

Chapter 48 — Parsing

0. What You Will Learn in This Chapter

Parsing means reading a string and understanding its structure.

Parsing problems appear in interviews when the input string represents:

- numbers
- expressions
- nested brackets
- encoded strings
- paths
- formulas
- logs
- versions
- arithmetic
- boolean expressions
- mini-languages

Parsing problems are usually not about one advanced algorithm. They are about careful state management.

They often use:

- index scanning
- stacks
- recursion
- tokenization
- sign handling
- operator precedence
- parentheses handling
- grammar-like thinking

By the end of this chapter, you should understand:

- What parsing is
- How to scan characters safely
- How to parse integers from strings
- How to handle signs and spaces
- How to parse nested brackets
- How to use stacks for expressions
- How to handle operator precedence
- How to parse parentheses recursively
- How to parse formulas
- How to parse file paths
- How to design recursive descent parsers
- How to avoid off-by-one and overflow bugs

Core problems covered in this chapter:

1. String to Integer atoi
2. Valid Number
3. Decode String
4. Basic Calculator
5. Basic Calculator II
6. Basic Calculator III
7. Evaluate Reverse Polish Notation

8. Simplify Path
 9. Mini Parser
 10. Number of Atoms
 11. Ternary Expression Parser
 12. Evaluate Boolean Binary Tree, preview
 13. Parsing boolean expression
 14. Compare Version Numbers
 15. Exclusive Time of Functions
 16. Reorder Data in Log Files
 17. Count of Atoms
 18. Lisp Expression Evaluation, advanced preview
-

1. What Is Parsing?

Parsing means converting a raw string into meaningful structure.

Example:

```
"123"
```

parses into integer:

```
123
```

Example:

```
"3 + 5 * 2"
```

parses into expression:

```
3 + (5 * 2)
```

Example:

```
"3[a2[c]]"
```

parses into:

```
accaccacc
```

Example:

```
"K4(ON(SO3)2)2"
```

parses into atom counts:

```
K4 O14 N2 S4
```

2. Why Parsing Problems Are Hard

Parsing problems are hard because of details:

- spaces
- signs
- nested structures
- multi-digit numbers
- operator precedence
- parentheses
- invalid input

- integer overflow
- recursive structure
- when to stop scanning
- when to push/pop stack

The algorithm may be simple, but implementation must be precise.

3. Common Parsing Patterns

Pattern	Used For
Linear scan	atoi, version compare
Stack of strings/numbers	Decode String
Stack of signs	Basic Calculator
Stack for precedence	Basic Calculator II
Recursive parsing	Calculator III, Mini Parser
Tokenization	logs, paths
Map stack	chemical formula
Reverse scan	ternary expression
Grammar parsing	Lisp, nested expressions

4. When to Use Parsing Techniques

Use parsing when string contains:

4.1 Numbers With Rules

Examples:

- atoi
- valid number
- compare versions

4.2 Nested Brackets

Examples:

- Decode String
- Mini Parser
- Number of Atoms

4.3 Arithmetic Expressions

Examples:

- Basic Calculator I, II, III
- Reverse Polish Notation

4.4 Paths or Logs

Examples:

- Simplify Path

- Reorder Logs
- Exclusive Time

4.5 Custom Grammar

Examples:

- Lisp Expression Evaluation
 - Boolean Expression Parser
-

5. When Not to Use Heavy Parsing

Do not overcomplicate when:

5.1 Simple Split Works

Example:

Compare Version Numbers can be parsed with two pointers or split.

5.2 Input Is Already Structured

If input is array/tree, use direct traversal.

5.3 Pattern Is Regular

If simple regex-like validation is enough, manual parsing may be simpler.

5.4 Problem Is Actually DP/String Matching

Do not confuse parsing with string DP or KMP.

6. Mental Models

6.1 Scanner

Use index i moving left to right.

At every step, decide what token starts at $s[i]$.

6.2 Token

A token is a meaningful unit:

- number
- operator
- parenthesis
- word
- comma
- bracket

6.3 Stack

Use stack when structure is nested or when you need to remember previous context.

6.4 Recursion

Use recursion when grammar is naturally nested.

Example:

expression inside parentheses

6.5 Precedence

Multiplication and division happen before addition and subtraction.

Use stack or recursive grammar.

7. Pattern Recognition Checklist

Ask:

19. Does the string represent numbers, expressions, or structure?
 20. Are there nested parentheses/brackets?
 21. Are there multi-digit numbers?
 22. Are spaces allowed?
 23. Are signs allowed?
 24. Is operator precedence involved?
 25. Is there recursion-like nesting?
 26. Can stack store previous context?
 27. Do I need tokenization?
 28. Are invalid inputs possible?
 29. Is overflow possible?
-

8. C++ Tools Needed

8.1 Character Checks

```
isdigit(c)
isalpha(c)
isalnum(c)
isspace(c)
```

8.2 Convert Digit

```
int d = c - '0';
```

8.3 Parse Number

```
long long num = 0;

while (i < s.size() && isdigit(s[i])) {
    num = num * 10 + (s[i] - '0');
    i++;
}
```

8.4 Stringstream

```
stringstream ss(sentence);
string word;

while (ss >> word) {
}
```

8.5 Stack

```
stack<int> st;
```

or:

```
vector<int> st;
```

9. Core Template 1 — Parse Integer From Index

```
long long parseNumber(string& s, int& i) {
    long long num = 0;

    while (i < s.size() && isdigit(s[i])) {
        num = num * 10 + (s[i] - '0');
        i++;
    }

    return num;
}
```

Important:

i is passed by reference so caller continues after number.

10. Core Template 2 — Skip Spaces

```
while (i < s.size() && s[i] == ' ') {
    i++;
}
```

Or:

```
isspace(s[i])
```

11. Core Template 3 — Stack for Nested Context

```
stack<string> stringStack;
stack<int> countStack;

string current;
int number = 0;

for (char c : s) {
    if (isdigit(c)) {
        number = number * 10 + (c - '0');
    } else if (c == '[') {
        countStack.push(number);
        stringStack.push(current);

        number = 0;
        current = "";
    } else if (c == ']') {
```

```

        int repeat = countStack.top();
        countStack.pop();

        string previous = stringStack.top();
        stringStack.pop();

        string expanded;

        while (repeat-- > 0) {
            expanded += current;
        }

        current = previous + expanded;
    } else {
        current.push_back(c);
    }
}

```

12. Core Template 4 — Calculator Stack for + - * /

```

vector<int> st;
char op = '+';
long long num = 0;

for each char plus sentinel:
    if digit:
        build num

    if operator or end:
        apply previous op to num
        reset num
        op = current operator

```

This works for Basic Calculator II.

13. Full Problem Walkthrough 1 — LeetCode 8: String to Integer atoi

13.1 Problem Summary

Convert string to 32-bit signed integer.

Rules:

30. Ignore leading spaces.
31. Optional sign.
32. Parse digits.

33. Stop at first non-digit.

34. Clamp to INT range.

13.2 C++ Code

```
class Solution {
public:
    int myAtoi(string s) {
        int i = 0;
        int n = s.size();

        while (i < n && s[i] == ' ') {
            i++;
        }

        int sign = 1;

        if (i < n && (s[i] == '+' || s[i] == '-')) {
            if (s[i] == '-') {
                sign = -1;
            }

            i++;
        }

        long long value = 0;

        while (i < n && isdigit(s[i])) {
            int digit = s[i] - '0';

            value = value * 10 + digit;

            if (sign == 1 && value > INT_MAX) {
                return INT_MAX;
            }

            if (sign == -1 && -value < INT_MIN) {
                return INT_MIN;
            }

            i++;
        }

        return sign * value;
    }
};
```

13.3 Common Mistakes

- Not skipping spaces
- Sign after digits
- Overflow

- Parsing after non-digit
-

14. Full Problem Walkthrough 2 — LeetCode 65: Valid Number

14.1 Problem Summary

Check if string is a valid decimal number.

Examples valid:

```
"2"  
"0089"  
"-0.1"  
"+3.14"  
"4."  
"- .9"  
"2e10"  
"-90E3"
```

Invalid:

```
"abc"  
"1a"  
"1e"  
"e3"  
"99e2.5"  
"--6"
```

14.2 Parsing Rules

A valid number can have:

- optional sign at beginning
- digits
- optional decimal point
- optional exponent part
- exponent may have optional sign
- exponent must have digits

14.3 State Flags

Track:

```
seenDigit  
seenDot  
seenExponent  
digitAfterExponent
```

14.4 C++ Code

```
class Solution {  
public:  
    bool isNumber(string s) {  
        bool seenDigit = false;  
        bool seenDot = false;
```

```

bool seenExponent = false;
bool digitAfterExponent = true;

for (int i = 0; i < s.size(); i++) {
    char c = s[i];

    if (isdigit(c)) {
        seenDigit = true;

        if (seenExponent) {
            digitAfterExponent = true;
        }
    } else if (c == '+' || c == '-') {
        if (i != 0 && s[i - 1] != 'e' && s[i - 1] != 'E') {
            return false;
        }
    } else if (c == '.') {
        if (seenDot || seenExponent) {
            return false;
        }

        seenDot = true;
    } else if (c == 'e' || c == 'E') {
        if (seenExponent || !seenDigit) {
            return false;
        }

        seenExponent = true;
        digitAfterExponent = false;
    } else {
        return false;
    }
}

return seenDigit && digitAfterExponent;
};

```

14.5 Key Lesson

Parsing validation often uses state flags.

15. Full Problem Walkthrough 3 — LeetCode 394: Decode String

15.1 Problem Summary

Decode encoded string.

Example:

```
"3[a2[c]]"
```

Output:

```
"accaccacc"
```

15.2 Pattern

Nested brackets.

Use stacks:

- count stack
- string stack

15.3 C++ Code

```
class Solution {
public:
    string decodeString(string s) {
        stack<int> countStack;
        stack<string> stringStack;

        string current;
        int number = 0;

        for (char c : s) {
            if (isdigit(c)) {
                number = number * 10 + (c - '0');
            } else if (c == '[') {
                countStack.push(number);
                stringStack.push(current);

                number = 0;
                current = "";
            } else if (c == ']') {
                int repeat = countStack.top();
                countStack.pop();

                string previous = stringStack.top();
                stringStack.pop();

                string expanded;

                for (int i = 0; i < repeat; i++) {
                    expanded += current;
                }

                current = previous + expanded;
            } else {
                current.push_back(c);
            }
        }
    }
};
```

```
        return current;
    }
};
```

15.4 Complexity

Time depends on output size.

$O(\text{output length})$

Space:

$O(n + \text{output nesting})$

16. Full Problem Walkthrough 4 — LeetCode 224: Basic Calculator

16.1 Problem Summary

Evaluate expression with:

- non-negative integers
- plus
- minus
- parentheses
- spaces

Example:

"(1+(4+5+2)-3)+(6+8)"

Answer:

23

16.2 Pattern

Use stack to store previous result and sign before parentheses.

16.3 C++ Code

```
class Solution {
public:
    int calculate(string s) {
        long long result = 0;
        long long number = 0;
        int sign = 1;

        stack<long long> st;

        for (int i = 0; i < s.size(); i++) {
            char c = s[i];

            if (isdigit(c)) {
                number = 0;

                while (i < s.size() && isdigit(s[i])) {
```

```

        number = number * 10 + (s[i] - '0');
        i++;
    }

    i--;

    result += sign * number;
} else if (c == '+') {
    sign = 1;
} else if (c == '-') {
    sign = -1;
} else if (c == '(') {
    st.push(result);
    st.push(sign);

    result = 0;
    sign = 1;
} else if (c == ')') {
    int prevSign = st.top();
    st.pop();

    long long prevResult = st.top();
    st.pop();

    result = prevResult + prevSign * result;
}
}

return result;
}
};

```

16.4 Why It Works

When entering parentheses:

previous result and sign are saved

When closing:

parentheses value is multiplied by saved sign and added to previous result

17. Full Problem Walkthrough 5 — LeetCode 227: Basic Calculator II

17.1 Problem Summary

Evaluate expression with:

- non-negative integers
- +, -, *, /

- spaces

No parentheses.

Example:

"3+2*2"

Answer:

7

17.2 Pattern

Multiplication and division have higher precedence.

Use stack:

- push positive number for +
- push negative number for -
- immediately apply * or / to stack top

17.3 C++ Code

```
class Solution {
public:
    int calculate(string s) {
        vector<long long> st;

        long long number = 0;
        char op = '+';

        for (int i = 0; i <= s.size(); i++) {
            char c = (i < s.size()) ? s[i] : '+';

            if (i < s.size() && c == ' ') {
                continue;
            }

            if (i < s.size() && isdigit(c)) {
                number = number * 10 + (c - '0');
            } else {
                if (op == '+') {
                    st.push_back(number);
                } else if (op == '-') {
                    st.push_back(-number);
                } else if (op == '*') {
                    st.back() = st.back() * number;
                } else if (op == '/') {
                    st.back() = st.back() / number;
                }

                op = c;
                number = 0;
            }
        }
    }
}
```

```

        long long result = 0;

        for (long long x : st) {
            result += x;
        }

        return result;
    }
};

```

17.4 Important Trick

Loop to `i <= s.size()` with sentinel operator to force processing last number.

18. Full Problem Walkthrough 6 — Basic Calculator III

18.1 Problem Summary

Evaluate expression with:

- +, -, *, /
- parentheses
- spaces

Example:

```
"2*(5+5*2)/3+(6/2+8)"
```

18.2 Pattern

Recursive parsing.

When you see (, recursively calculate inside.

When you see), return current result.

18.3 C++ Code

```

class Solution {
public:
    int calculate(string s) {
        int index = 0;
        return helper(s, index);
    }

    int helper(string& s, int& i) {
        vector<long long> st;

        long long number = 0;
        char op = '+';

        while (i < s.size()) {
            char c = s[i];

            if (c == ' ') {

```

```

        i++;
        continue;
    }

    if (isdigit(c)) {
        number = 0;

        while (i < s.size() && isdigit(s[i])) {
            number = number * 10 + (s[i] - '0');
            i++;
        }

        continue;
    }

    if (c == '(') {
        i++;
        number = helper(s, i);
        continue;
    }

    if (c == '+' || c == '-' || c == '*' || c == '/' || c ==
')) {
        apply(st, op, number);

        op = c;
        number = 0;

        i++;

        if (c == ')') {
            break;
        }
    }

    apply(st, op, number);

    long long result = 0;

    for (long long x : st) {
        result += x;
    }

    return result;
}

void apply(vector<long long>& st, char op, long long number) {
    if (op == '+') {

```

```

        st.push_back(number);
    } else if (op == '-') {
        st.push_back(-number);
    } else if (op == '*') {
        st.back() *= number;
    } else if (op == '/') {
        st.back() /= number;
    }
}
};

```

18.4 Note

Calculator III implementations are easy to get wrong.

A cleaner production version separates tokenization and parsing, but this interview-style recursive version is common.

19. Full Problem Walkthrough 7 — LeetCode 150: Evaluate Reverse Polish Notation

19.1 Problem Summary

Evaluate expression in Reverse Polish Notation.

Example:

```
["2", "1", "+", "3", "*"]
```

means:

```
(2 + 1) * 3 = 9
```

19.2 Pattern

Use stack.

When token is number, push.

When operator, pop two operands.

19.3 C++ Code

```

class Solution {
public:
    int evalRPN(vector<string>& tokens) {
        stack<long long> st;

        for (string token : tokens) {
            if (token == "+" || token == "-" || token == "*" ||
token == "/" ) {
                long long b = st.top();
                st.pop();

                long long a = st.top();

```

```

        st.pop();

        if (token == "+") {
            st.push(a + b);
        } else if (token == "-") {
            st.push(a - b);
        } else if (token == "*") {
            st.push(a * b);
        } else {
            st.push(a / b);
        }
    } else {
        st.push(stoll(token));
    }
}

return st.top();
}
};

```

19.4 Common Mistake

Order matters for subtraction and division.

a - b
a / b

not:

b - a
b / a

20. Full Problem Walkthrough 8 — LeetCode 71: Simplify Path

20.1 Problem Summary

Given Unix path, simplify it.

Rules:

"." means current directory
 ".." means parent directory
 multiple slashes treated as one

Example:

"/a/./b/../../../../c/"

Output:

"/c"

20.2 Pattern

Split by /.

Use stack of directory names.

20.3 C++ Code

```
class Solution {
public:
    string simplifyPath(string path) {
        vector<string> st;

        int i = 0;
        int n = path.size();

        while (i < n) {
            while (i < n && path[i] == '/') {
                i++;
            }

            string token;

            while (i < n && path[i] != '/') {
                token.push_back(path[i]);
                i++;
            }

            if (token.empty() || token == ".") {
                continue;
            }

            if (token == "..") {
                if (!st.empty()) {
                    st.pop_back();
                }
            } else {
                st.push_back(token);
            }
        }

        string result;

        for (string dir : st) {
            result += "/";
            result += dir;
        }

        return result.empty() ? "/" : result;
    }
};
```

21. Full Problem Walkthrough 9 — LeetCode 385: Mini Parser

21.1 Problem Summary

Parse nested integer list.

Example:

```
"324"
```

returns integer.

Example:

```
"[123,[456,[789]]]"
```

returns nested list.

21.2 Pattern

Nested brackets.

Use stack.

21.3 High-Level Code

LeetCode provides NestedInteger.

```
class Solution {
public:
    NestedInteger deserialize(string s) {
        if (s[0] != '[') {
            return NestedInteger(stoi(s));
        }

        stack<NestedInteger> st;
        int i = 0;

        while (i < s.size()) {
            if (s[i] == '[') {
                st.push(NestedInteger());
                i++;
            } else if (s[i] == ']') {
                NestedInteger current = st.top();
                st.pop();

                if (st.empty()) {
                    return current;
                }

                st.top().add(current);
                i++;
            } else if (s[i] == ',') {
                i++;
            } else {
                int sign = 1;
```

```

        if (s[i] == '-') {
            sign = -1;
            i++;
        }

        int number = 0;

        while (i < s.size() && isdigit(s[i])) {
            number = number * 10 + (s[i] - '0');
            i++;
        }

        st.top().add(NestedInteger(sign * number));
    }

    return NestedInteger();
};

```

22. Full Problem Walkthrough 10 — LeetCode 726: Number of Atoms

22.1 Problem Summary

Given chemical formula, count atoms.

Example:

"K4(ON(SO3)2)2"

Output:

"K4N2O14S4"

22.2 Parsing Rules

- Atom starts with uppercase letter.
- It may have lowercase letters.
- Number after atom means count.
- Parentheses group atoms.
- Number after group multiplies whole group.

22.3 Pattern

Use stack of maps.

Each parentheses level has a map.

22.4 C++ Code

```

class Solution {
public:
    string countOfAtoms(string formula) {
        stack<map<string, int>> st;
    }
};

```

```

    st.push(map<string, int>());

    int i = 0;
    int n = formula.size();

    while (i < n) {
        if (formula[i] == '(') {
            st.push(map<string, int>());
            i++;
        } else if (formula[i] == ')') {
            i++;

            int multiplier = parseNumber(formula, i);

            if (multiplier == 0) {
                multiplier = 1;
            }

            map<string, int> top = st.top();
            st.pop();

            for (auto& [atom, count] : top) {
                st.top()[atom] += count * multiplier;
            }
        } else {
            string atom = parseAtom(formula, i);

            int count = parseNumber(formula, i);

            if (count == 0) {
                count = 1;
            }

            st.top()[atom] += count;
        }
    }

    string result;

    for (auto& [atom, count] : st.top()) {
        result += atom;

        if (count > 1) {
            result += to_string(count);
        }
    }

    return result;
}

```

```

string parseAtom(string& s, int& i) {
    string atom;

    atom.push_back(s[i]);
    i++;

    while (i < s.size() && islower(s[i])) {
        atom.push_back(s[i]);
        i++;
    }

    return atom;
}

int parseNumber(string& s, int& i) {
    int number = 0;

    while (i < s.size() && isdigit(s[i])) {
        number = number * 10 + (s[i] - '0');
        i++;
    }

    return number;
}
};

```

22.5 Why map?

Output must be sorted lexicographically by atom name.
map keeps keys sorted.

23. Full Problem Walkthrough 11 — LeetCode 439: Ternary Expression Parser

23.1 Problem Summary

Parse nested ternary expression.

Example:

"T?2:3"

Answer:

"2"

Example:

"F?1:T?4:5"

Answer:

"4"

23.2 Pattern

Ternary is right-associative.

Parse from right to left using stack.

23.3 C++ Code

```
class Solution {
public:
    string parseTernary(string expression) {
        stack<char> st;

        for (int i = expression.size() - 1; i >= 0; i--) {
            char c = expression[i];

            if (!st.empty() && st.top() == '?') {
                st.pop();

                char trueExpr = st.top();
                st.pop();

                st.pop();

                char falseExpr = st.top();
                st.pop();

                if (c == 'T') {
                    st.push(trueExpr);
                } else {
                    st.push(falseExpr);
                }
            } else if (c != ':') {
                st.push(c);
            }
        }

        return string(1, st.top());
    }
};
```

23.4 Why Right to Left?

Ternary expressions group from right to left.

Parsing from right naturally resolves innermost ternary first.

24. Full Problem Walkthrough 12 — LeetCode 1106: Parsing A Boolean Expression

24.1 Problem Summary

Parse expression with:

t, f, !, &, |

Examples:

```
"!(f)" -> true
"|(f,t)" -> true
"&(t,f)" -> false
```

24.2 Pattern

Use stack.

When encountering), evaluate back to matching (.

24.3 C++ Code

```
class Solution {
public:
    bool parseBoolExpr(string expression) {
        stack<char> st;

        for (char c : expression) {
            if (c == ',') {
                continue;
            }

            if (c != ')') {
                st.push(c);
            } else {
                int trueCount = 0;
                int falseCount = 0;

                while (st.top() != '(') {
                    char val = st.top();
                    st.pop();

                    if (val == 't') {
                        trueCount++;
                    } else if (val == 'f') {
                        falseCount++;
                    }
                }

                if (st.top() == '(') {
                    st.pop();
                }

                if (trueCount > 0) {
                    st.push('t');
                } else {
                    st.push('f');
                }
            }
        }

        return st.top() == 't';
    }
};
```

```

        char op = st.top();
        st.pop();

        char result;

        if (op == '!') {
            result = falseCount == 1 ? 't' : 'f';
        } else if (op == '&') {
            result = falseCount == 0 ? 't' : 'f';
        } else {
            result = trueCount > 0 ? 't' : 'f';
        }

        st.push(result);
    }

    return st.top() == 't';
}
};

```

25. Full Problem Walkthrough 13 — LeetCode 636: Exclusive Time of Functions

25.1 Problem Summary

Given logs:

```
function_id:start_or_end:timestamp
```

Return exclusive execution time for each function.

Nested function calls pause parent.

25.2 Pattern

Stack of active function IDs.

Track previous timestamp.

25.3 C++ Code

```

class Solution {
public:
    vector<int> exclusiveTime(int n, vector<string>& logs) {
        vector<int> result(n, 0);
        stack<int> st;

        int prevTime = 0;

        for (string log : logs) {

```

```

        auto parts = split(log);

        int id = stoi(parts[0]);
        string type = parts[1];
        int time = stoi(parts[2]);

        if (type == "start") {
            if (!st.empty()) {
                result[st.top()] += time - prevTime;
            }

            st.push(id);
            prevTime = time;
        } else {
            result[st.top()] += time - prevTime + 1;
            st.pop();

            prevTime = time + 1;
        }
    }

    return result;
}

vector<string> split(string s) {
    vector<string> parts;
    string current;

    for (char c : s) {
        if (c == ':') {
            parts.push_back(current);
            current.clear();
        } else {
            current.push_back(c);
        }
    }

    parts.push_back(current);

    return parts;
}
};

```

25.4 Key Detail

End timestamp is inclusive.

So add:

```
time - prevTime + 1
```

Then set:

```
prevTime = time + 1
```

26. Full Problem Walkthrough 14 — Reorder Data in Log Files

26.1 Problem Summary

Logs have identifier and content.

Letter logs come before digit logs.

Letter logs sorted by content, then identifier.

Digit logs remain original order.

26.2 Pattern

Custom comparator + parsing identifier/content.

26.3 C++ Code

```
class Solution {
public:
    vector<string> reorderLogFiles(vector<string>& logs) {
        stable_sort(logs.begin(), logs.end(), [](const string& a,
const string& b) {
            int posA = a.find(' ');
            int posB = b.find(' ');

            string idA = a.substr(0, posA);
            string idB = b.substr(0, posB);

            string contentA = a.substr(posA + 1);
            string contentB = b.substr(posB + 1);

            bool isDigitA = isdigit(contentA[0]);
            bool isDigitB = isdigit(contentB[0]);

            if (isDigitA && isDigitB) {
                return false;
            }

            if (isDigitA) {
                return false;
            }

            if (isDigitB) {
                return true;
            }

            if (contentA == contentB) {
                return idA < idB;
            }
        });
    }
};
```

```
        return contentA < contentB;
    });

    return logs;
}
};
```

27. Full Problem Walkthrough 15 — Compare Version Numbers Review

27.1 Problem Summary

Compare version strings like:

```
"1.01"
"1.001"
```

Parts are numeric.

Leading zeros ignored.

27.2 C++ Code

```
class Solution {
public:
    int compareVersion(string version1, string version2) {
        int i = 0;
        int j = 0;

        int n = version1.size();
        int m = version2.size();

        while (i < n || j < m) {
            long long num1 = 0;
            long long num2 = 0;

            while (i < n && version1[i] != '.') {
                num1 = num1 * 10 + (version1[i] - '0');
                i++;
            }

            while (j < m && version2[j] != '.') {
                num2 = num2 * 10 + (version2[j] - '0');
                j++;
            }

            if (num1 < num2) {
                return -1;
            }
        }
    }
};
```

```
        if (num1 > num2) {
            return 1;
        }

        i++;
        j++;
    }

    return 0;
};
```

28. Recursive Descent Parsing

Preview

28.1 What Is Recursive Descent?

Recursive descent is a parsing method where each grammar rule becomes a function.

Example grammar:

```
expr  = term ((+ | -) term)*
term  = factor ((* | /) factor)*
factor = number | '(' expr ')'
```

Functions:

```
parseExpression()
parseTerm()
parseFactor()
```

This cleanly handles precedence.

28.2 When Useful

Use recursive descent for:

- full calculators
- expression languages
- nested grammar problems
- Lisp-like expressions

28.3 Why Not Always Use It?

It is longer to implement.

For interview calculator problems, stack-based methods are usually faster to write.

29. Common Parsing Edge Cases

29.1 Empty String

Check if valid input can be empty.

29.2 Spaces

Skip where allowed.

29.3 Multi-Digit Numbers

Do not parse digit by digit as separate numbers.

29.4 Negative Numbers

Distinguish unary minus from binary minus.

29.5 Parentheses

Always ensure push/pop matches.

29.6 Integer Overflow

Use long long while parsing.

29.7 End of String

Use sentinel operator or process final number manually.

30. Common Parsing Mistakes

Mistake 1 — Forgetting Last Number

Calculator II often misses final number without sentinel.

Mistake 2 — Wrong Division Order

In RPN:

a / b

not:

b / a

Mistake 3 — Not Handling Spaces

Spaces can appear anywhere in calculators.

Mistake 4 — Stack Underflow

Always ensure stack has expected values.

Mistake 5 — Inclusive End Time Bug

Exclusive Time logs use inclusive end timestamps.

Mistake 6 — Wrong Parentheses Context

For Basic Calculator I, save both result and sign.

31. Interview Explanation Templates

31.1 atoi

I skip spaces, parse optional sign, then read digits while checking overflow before returning the clamped value.

31.2 Decode String

I use one stack for repeat counts and one stack for previous strings. When I close a bracket, I repeat the current string and append it to the previous context.

31.3 Basic Calculator I

I maintain current result and sign. On opening parenthesis, I push previous result and sign. On closing, I combine the parenthesized result with the saved context.

31.4 Basic Calculator II

I use a stack to handle precedence. Addition and subtraction push signed numbers, while multiplication and division immediately update the stack top.

31.5 Chemical Formula

I use a stack of maps. Each parenthesized group has its own map. When the group closes, I multiply its counts and merge into the previous level.

32. Complexity Patterns

Problem	Time	Space
atoi	$O(n)$	$O(1)$
Valid Number	$O(n)$	$O(1)$
Decode String	$O(\text{output length})$	$O(n)$
Basic Calculator I	$O(n)$	$O(n)$
Basic Calculator II	$O(n)$	$O(n)$
Basic Calculator III	$O(n)$	$O(n)$
RPN	$O(n)$	$O(n)$
Simplify Path	$O(n)$	$O(n)$
Mini Parser	$O(n)$	$O(n)$
Number of Atoms	$O(n \log k)$	$O(n)$
Ternary Parser	$O(n)$	$O(n)$
Boolean Parser	$O(n)$	$O(n)$
Exclusive Time	$O(\log s * \log \text{length})$ roughly	$O(n)$

33. Expanded Practice Set

33.1 Must-Do Medium

- 35. String to Integer atoi
- 36. Decode String
- 37. Basic Calculator
- 38. Basic Calculator II
- 39. Evaluate Reverse Polish Notation
- 40. Simplify Path
- 41. Compare Version Numbers
- 42. Reorder Data in Log Files
- 43. Exclusive Time of Functions

33.2 Strong Interview Practice

- 44. Valid Number
- 45. Mini Parser

- 46. Number of Atoms
- 47. Ternary Expression Parser
- 48. Parsing A Boolean Expression
- 49. Basic Calculator III
- 50. Lisp Expression Evaluation

33.3 Hard / Stretch

- 51. Lisp Expression Evaluation
 - 52. Basic Calculator IV
 - 53. Chemical Formula Parser variants
 - 54. Recursive Descent Calculator
 - 55. Expression Add Operators, backtracking + parsing
 - 56. Serialize and Deserialize Nested Structures
-

34. Suggested Study Order

Step 1 — Linear Parsing

Solve:

- 57. atoi
- 58. Compare Version Numbers
- 59. Valid Number

Goal:

Master index scanning and state flags.

Step 2 — Stack Parsing

Solve:

- 60. Decode String
- 61. Simplify Path
- 62. Evaluate RPN

Goal:

Learn stack context.

Step 3 — Calculators

Solve:

- 63. Basic Calculator
- 64. Basic Calculator II
- 65. Basic Calculator III

Goal:

Master signs, precedence, and parentheses.

Step 4 — Structured Parsing

Solve:

- 66. Mini Parser
- 67. Number of Atoms
- 68. Boolean Expression Parser

Goal:

Handle nested structures.

Step 5 — Logs and Custom Sort

Solve:

- 69. Exclusive Time
- 70. Reorder Log Files

Goal:
Parse structured records.

35. Pattern Decision Table

Problem Clue	Parsing Strategy
convert string to int	linear scan + overflow
validate decimal/scientific number	state flags
nested brackets with repeat	count stack + string stack
+ and - with parentheses	result/sign stack
+ - * / no parentheses	stack immediate * /
full arithmetic with parentheses	recursion + stack
reverse polish notation	operand stack
unix path	split by slash + stack
nested integer list	stack / recursion
chemical formula	stack of maps
ternary expression	reverse scan stack
boolean expression	stack evaluate at ')
function logs	stack + previous timestamp
version numbers	parse segments

36. Mini Problem 1 — Parse CSV-Like Tokens

If delimiter is comma:

```
vector<string> splitByComma(string s) {  
    vector<string> result;  
    string current;  
  
    for (char c : s) {  
        if (c == ',') {  
            result.push_back(current);  
            current.clear();  
        } else {
```

```

        current.push_back(c);
    }
}

result.push_back(current);

return result;
}

```

37. Mini Problem 2 — Parse Signed Integer From Index

```

long long parseSignedInteger(string& s, int& i) {
    int sign = 1;

    if (i < s.size() && (s[i] == '+' || s[i] == '-')) {
        if (s[i] == '-') {
            sign = -1;
        }

        i++;
    }

    long long number = 0;

    while (i < s.size() && isdigit(s[i])) {
        number = number * 10 + (s[i] - '0');
        i++;
    }

    return sign * number;
}

```

38. Mini Problem 3 — Remove Outer Parentheses

Track depth.

```

class Solution {
public:
    string removeOuterParentheses(string s) {
        string result;
        int depth = 0;

        for (char c : s) {

```

```

        if (c == '(') {
            if (depth > 0) {
                result.push_back(c);
            }

            depth++;
        } else {
            depth--;

            if (depth > 0) {
                result.push_back(c);
            }
        }
    }

    return result;
}
};

```

39. Mini Problem 4 — Maximum Nesting Depth of Parentheses

```

class Solution {
public:
    int maxDepth(string s) {
        int depth = 0;
        int answer = 0;

        for (char c : s) {
            if (c == '(') {
                depth++;
                answer = max(answer, depth);
            } else if (c == ')') {
                depth--;
            }
        }

        return answer;
    }
};

```

40. Common Edge Case Checklist

Before submitting parsing code, test:

```
empty string
only spaces
leading spaces
trailing spaces
multi-digit numbers
negative numbers
plus sign
nested parentheses
deep nesting
operator at end if invalid possible
division with negative values
path with multiple slashes
path with ..
formula with nested groups
logs with same timestamp
very large number
```

41. Debugging Parsing Problems

If parsing fails, check:

71. Is index incremented exactly once per character?
 72. Are multi-digit numbers parsed correctly?
 73. Are spaces skipped in all relevant places?
 74. Is final token processed?
 75. Are stack push/pop operations balanced?
 76. Is sign applied at correct time?
 77. Is operator precedence handled?
 78. Are parentheses handled recursively or by stack?
 79. Is overflow handled?
 80. Are end timestamps inclusive or exclusive?
-

42. What to Master Before Chapter 49

Before moving to Bit Manipulation, make sure you can:

- Parse integers from strings
- Handle signs and overflow
- Solve atoi
- Solve Decode String
- Solve Basic Calculator I
- Solve Basic Calculator II
- Understand Basic Calculator III
- Solve Evaluate RPN
- Solve Simplify Path
- Solve Number of Atoms
- Solve Ternary Expression Parser
- Parse structured logs
- Use stack for nested contexts
- Explain operator precedence handling

43. Final Cheat Sheet

Parse Number

```
while (i < n && isdigit(s[i])) {  
    number = number * 10 + (s[i] - '0');  
    i++;  
}
```

Skip Spaces

```
while (i < n && s[i] == ' ') {  
    i++;  
}
```

Decode String

```
digit -> build count  
 '[' -> push count and current string  
 ']' -> pop and expand  
 char -> append
```

Calculator II

```
+ -> push num  
- -> push -num  
* -> stack.top *= num  
/ -> stack.top /= num
```

Parentheses Calculator

```
push previous result and sign on '('  
combine on ')'
```

Chemical Formula

```
'(' -> push new map  
)' -> pop map, multiply, merge  
atom -> parse name and count
```

Exclusive Time

```
start: add time - prevTime to current function  
end: add time - prevTime + 1
```

44. Core Takeaway

Parsing is controlled scanning.

The key is to know what the next token is and how it changes your state.

Most parsing problems use:

- an index
- a stack

- a current number/string
- a current sign/operator
- recursive handling for nested structures

For calculators, focus on:

sign
operator precedence
parentheses

For nested structures, focus on:

push context when entering
pop and merge when exiting

Parsing problems are detail-heavy, so always test edge cases.